

和豆包的对话_0630

ZeroFairyClient

迭代优化版完整开发框架（融合昔涟优秀架构，剔除全自动游戏日常，保留轻量化RAG、记忆、插件-工具链）

一、项目定位更新

核心：绝区零Fairy拟人桌面AI客户端

1. 核心主线：Live2D情绪神态交互、语音通话、Fairy专属强约束人设对话、BYOK多模型插件适配
2. 新增拓展能力（参考昔涟优化）：分层防膨胀Prompt架构、轻量化RAG游戏知识库、Function Call通用工具系统、MCP插件扩展、长期记忆&用户画像、文档处理工具（Excel/PDF/Word/PPT）、-联网搜索、定时提醒（无全自动脚本）
3. 剔除模块：全自动游戏日常自动化批量脚本

二、完整整合技术栈（原有栈+昔涟新增优化技术点）

1. Electron桌面基座层（沿用+优化进程分层）

- Electron + Electron-Vite + TypeScript 全栈
- electron-builder 打包、electron-store（密钥加密存储）、crypto 本地加密
- sqlite3：聊天记录、长期记忆、知识库分片持久化
- winston 分层日志

优化架构参考昔涟：严格拆分职责

- 主进程：承载全部重运算逻辑（LLM调度、RAG检索、工具调用、记忆系统、TTS、MCP插件、权限控制、定时提醒）

- 渲染进程：仅负责纯可视化UI、Live2D画布、语音通话界面、聊天展示，不运行任何大模型/检索-逻辑，杜绝动画卡顿

2. 前端可视化渲染层（完全自主设计赛博UI）

- Vue3 + Pinia全局状态 + VueRouter
- NaiveUI + TailwindCSS + SCSS 自定义磨砂赛博界面
- Iconify机械风图标
- PixiJS + pixi-live2d-display 驱动Live2D角色画布
- Web Audio API（音频振幅采集）、Howler.js音频播放
- Web Speech API 麦克风ASR语音识别

新增：AG-UI标准化事件流

模型回复、工具执行、情绪状态、语音状态、RAG检索结果全部封装事件流，统一推送前端，替代零-散IPC监听，通信逻辑更整洁

3. LLM多模型适配器（「先造车后接司机」插件架构，参考昔涟适-配方案）

- Axios SSE流式对话打字机输出
- 统一抽象基类，双标准兼容：OpenAI-compatible / Anthropic-like
- 独立厂商插件目录，新增模型仅新增适配文件，不改动核心业务代码
- 统一封装Function Call工具调用解析，抹平各家模型函数调用格式差异

4. AI核心增强模块（昔涟核心优化点全部加入，剔除重型自动化）

1. 分层防Token膨胀Prompt架构（核心优化）

四层隔离分层，自动裁剪过期上下文，解决多轮对话人设跑偏、token超限、注意力稀释：

- 底层固定层：不可修改Fairy完整人设、世界观约束、角色称呼规则
- 知识召回层：RAG检索到的绝区零游戏知识库片段（动态插入，用完自动清理）
- 对话上下文层：滚动窗口，自动淘汰超期历史对话
- 工具指令层：临时Function Call调用规则，单次工具执行后销毁

2. 轻量化RAG检索系统（剔除超大办公文档重型逻辑，适配游戏文本）

- Worldbook游戏知识库：导入绝区零剧情、代理人、空洞、怪物、装备资料分片存储
- 检索召回+重排逻辑：关键词召回、相似度排序，过滤无关文本，减少无效token
- 支持导入小型文档（角色设定PDF、剧情Word），不支持百页级超长办公文档处理

3. 长期记忆&用户画像系统

自动沉淀长期聊天信息：玩家游戏偏好、对话习惯、常问的游戏内容，存入sqlite记忆库，每次对话自动检索注入上下文，贴合玩家习惯

4. MCP插件扩展系统

可插拔技能插件架构，后续新增游戏辅助工具（配装查询、活动日历、角色强度查询）无需重构底层代码

5. 通用工具链（保留轻量化生产力，删除全自动游戏脚本）

内置Function Call可调用工具列表：

- 文档处理：Excel/PPT/PDF/Word生成、文本翻译
- 信息获取：联网搜索、实时天气、汇率查询、新闻检索
- 本地操作：文件读写、本地图片读取
- 生活辅助：记账、出行规划、定时弹窗提醒（仅弹窗通知，无批量自动执行任务）

6. Live2D角色交互模块（你的原有核心，优先级最高）

- PixiJS驱动Live2D Fairy模型，独立可悬浮、置顶、拖动窗口
- 情绪联动系统：LLM返回情绪标签自动切换眼部、眉眼神态
- 空闲待机逻辑：无交互3s后自主左右扫视、缓慢眨眼模拟观察用户
- TTS音频振幅联动：说话时眼球细微动态，同步嘴型神态

7. 语音通话模块

云端TTS（ElevenLabs角色声线克隆/火山语音）、麦克风实时语音识别、通话独立可视化弹窗

三、优化后完整项目目录结构（整合新增模块）

```
ZeroFairyClient/
├── electron/          # Electron主进程（全部重运算逻辑放这里）
│   ├── main/
│   │   ├── index.ts    # 程序入口、托盘、窗口管理、定时任务调度
│   │   ├── store/      # 加密密钥、全局配置持久化
│   │   ├── db/         # sqlite数据库
│   │   │   ├── chatHistory.ts # 聊天记录
│   │   │   ├── memoryBase.ts # 用户长期记忆、画像
│   │   │   ├── gameWorldBook.ts # 绝区零游戏知识库RAG库
│   │   ├── llmAdapter/  # 多模型插件适配器（先造车后接司机核心）
│   │   │   ├── baseModel.ts # 统一抽象基类
│   │   │   ├── providers/  # 各厂商独立插件
│   │   │   ├── promptCore.ts # 四层分层防膨胀Prompt架构
│   │   │   ├── streamHandler.ts # SSE流式解析
│   │   │   ├── functionCall.ts # 统一函数调用解析抹平厂商差异
│   │   ├── ragEngine/   # 轻量化RAG检索召回重排模块
│   │   │   ├── chunkSplit.ts # 游戏文本分片（轻量化，不处理超大文档）
│   │   │   ├── retrieveRank.ts # 检索、相似度重排
│   │   ├── agentEventBus.ts # AG-UI全局事件流总线
│   │   ├── toolSystem/  # MCP可插拔工具插件系统
│   │   │   ├── baseTool.ts # 工具抽象基类
│   │   │   ├── plugins/   # 所有工具插件（文档/搜索/记账/定时提醒等）
│   │   ├── memorySystem/ # 长期记忆、用户画像管理
│   │   ├── voice/       # TTS语音合成、音频处理
│   │   ├── timerNotify.ts # 轻量化定时弹窗提醒（无自动化脚本）
│   ├── preload/        # IPC通信桥，隔离主/渲染进程权限
│   └── ipcBridge.ts
├── src/                # Vue渲染进程（仅UI展示，无重计算）
│   ├── main.ts
│   ├── router/
│   ├── stores/         # Pinia全局状态（接收事件流更新状态）
│   │   ├── llmStore.ts
│   │   ├── chatStore.ts
│   │   ├── live2dStore.ts # Live2D情绪、待机状态
│   │   ├── voiceStore.ts
│   │   ├── toolStore.ts  # 工具插件面板状态
│   │   ├── ragStore.ts   # 知识库管理页面状态
│   ├── components/
│   └── ChatBox/        # 流式打字聊天框
```

```
| | |—— ConfigPanel/ # API密钥、模型参数配置
| | |—— VoiceCall/ # 语音通话弹窗
| | |—— Live2dCanvas/ # Live2D画布、情绪/待机控制器
| | |—— ToolManage/ # MCP工具插件管理面板
| | |—— WorldBookManage/ # 游戏知识库导入管理页面
| |—— views/
| | |—— IndexChat.vue # 主聊天页面
| | |—— Config.vue # 全局配置页
| | |—— VoiceCall.vue # 语音通话界面
| | |—— WorldBook.vue # RAG游戏知识库管理页
| | |—— ToolPlugin.vue # 工具插件配置页
| | |—— MemoryView.vue # 用户长期记忆查看页
| |—— assets/
| |—— utils/
| | |—— eventClient.ts # 前端监听AG-UI事件流工具
| | |—— audioAnalyse.ts
|—— .electron-vite
|—— electron-builder.json
|—— package.json
```

四、四大核心业务完整流程（优化整合版）

流程1：LLM分层Prompt+RAG对话主链路（解决人设崩坏、Token-溢出）

1. 用户在前端发送文字/语音转文字提问，通过IPC推送事件到主进程事件总线
2. 主进程执行三步前置处理：
 - ① memorySystem检索用户长期记忆、游戏偏好，注入临时上下文
 - ② ragEngine检索Worldbook绝区零知识库，召回匹配剧情/角色片段
 - ③ promptCore四层拼接完整请求文本（固定人设层+RAG知识层+历史对话层+工具指令层），自动-裁剪过期对话控制总token长度
3. llm适配器根据用户选择的厂商API Key转发请求，开启SSE流式输出
4. 解析模型返回内容：

- 识别[emotion:xxx]情绪标签，通过事件流推送前端同步Live2D神态
- 识别Function Call工具调用指令，自动调度toolSystem执行对应插件（联网/文档生成/翻译等） - ，工具结果回注入对话继续推理

5. 流式文字逐段通过事件流推送到前端聊天框打字机渲染

6. 对话结束自动更新memorySystem长期记忆库，沉淀本次对话关键信息

流程2：Live2D神态+待机交互链路（核心特色，不变）

1. 全局计时器监听用户所有交互（输入、点击、语音开启），每次操作重置倒计时
2. 3秒无交互触发待机动画状态机：缓慢左右扫视、随机眨眼、轻微头部晃动模拟观察用户
3. 接收事件流推送的情绪标签，中断待机，切换对应眼部/眉眼动画，动画时长绑定当前TTS语音时长
4. TTS播放时实时采集音频振幅，叠加细微眼球动态，说话神态更自然

流程3：语音通话完整链路

1. 打开语音通话页面，开启麦克风Web Speech实时转文字提问
2. 执行上面完整LLM对话链路，生成带情绪标签的Fairy台词
3. 台词送入云端TTS生成专属声线音频
4. 音频加载播放，同步推送事件流更新Live2D情绪神态+振幅联动眼部动态
5. 通话全程Live2D悬浮窗口置顶，随时可见角色神态变化

流程4：RAG游戏知识库&工具插件拓展链路

1. 用户在WorldBook页面导入绝区零剧情PDF/角色设定文档，ragEngine轻量化分片存入本地sqlite知识库
2. 聊天提问时自动检索匹配游戏资料，避免AI编造错误游戏设定
3. 用户可在ToolPlugin页面启用/关闭各类工具插件，模型自动按需调用；支持后续自定义新增游戏辅助-插件，无需修改底层代码
4. 轻量化定时提醒工具：设置游戏活动、日程弹窗提醒，仅弹出通知，无全自动后台执行脚本

五、分阶段迭代开发流程（优化后，循序渐进，降低压力-）

阶段1：基础桌面+多模型适配器核心（3天）

1. 搭建Electron-Vite+TS工程，搭建空白基础UI页面
2. 实现主/渲染进程严格分层架构、AG-UI事件总线通信
3. 开发多厂商插件化适配器、SSE流式对话、API密钥加密存储
4. 完成基础单层对话，验证「先造车后接司机」可插拔模型架构

阶段2：四层分层Prompt+轻量化RAG知识库（3天，核心优化点）

1. 开发promptCore四层防膨胀架构，实现上下文自动滚动裁剪
2. 实现轻量化ragEngine、Worldbook游戏知识库导入、检索重排逻辑
3. 开发长期记忆&用户画像系统，对话自动沉淀记忆

阶段3：MCP工具插件系统（2天，轻量化生产力工具）

1. 编写工具抽象基类，搭建可插拔MCP插件架构
2. 开发基础工具：联网搜索、文档生成、翻译、记账、轻量化定时弹窗提醒
3. 完成Function Call统一解析，抹平各大模型函数调用差异

阶段4：语音通话模块（2天）

1. 语音通话可视化弹窗、麦克风ASR语音识别
2. 对接云端TTS、音频播放、音频振幅采集工具类

阶段5：Live2D动态角色完整交互（3天，产品核心卖点）

1. Pixi+Pixi-Live2D画布搭建独立悬浮角色窗口
2. 实现情绪标签联动眼部神态动画、语音振幅同步动态

3. 开发无交互自主观察待机状态机、计时器控制逻辑

阶段6：UI完整美化、配套管理页面（2天）

1. 全局自定义赛博磨砂玻璃UI、动效、全部页面视觉打磨
2. 开发知识库管理页、工具插件配置页、记忆查看页
3. 完善窗口拖拽、置顶、托盘后台常驻、开机自启等桌面功能

阶段7：测试、打包发布（2天）

1. 全流程联调：对话、RAG检索、工具调用、语音、Live2D动画同步测试
2. 修复token溢出、人设跑偏、多模型适配bug
3. 配置electron-builder，打包Windows exe安装包

六、优化后项目差异化优势

1. **底层架构更稳定**：借鉴昔涟主进程承载全部重运算逻辑，彻底解决Live2D、语音播放卡顿问题，标准化事件流简化前后端通信
2. **解决长期痛点**：四层分层Prompt+RAG记忆系统，杜绝多轮对话后Fairy人设崩坏、游戏设定乱编、-token超限报错
3. **拓展性极强**：MCP插件架构，后续新增任何游戏辅助功能（配装查询、活动日历）不用重构底层核心代码
4. **功能取舍精准**：只保留对你有用的轻量化生产力工具、游戏知识库检索，剔除重型办公文档处理、全自动游戏脚本，不增加无意义开发工作量
5. **完全自研UI视觉**，核心体验聚焦Fairy拟人神态交互，工具、知识库仅作为配套辅助功能，不本末倒置

(豆包AI生成)